

Much Ado about Prompting:

LLM Classification of Text Messages from Experiments

Can Çelebi*¹ and Stefan P. Penczynski²

¹*Vienna Center for Experimental Economics (VCEE), University of Vienna, Oskar-Morgenstern-Platz 1, 1090 Vienna, Austria. Email: can.celebi@univie.ac.at*

²*School of Economics and Centre for Behavioural and Experimental Social Science (CBESS), University of East Anglia, Norwich Research Park, Norwich NR4 7TJ, United Kingdom. Email: s.penczynski@uea.ac.uk, Tel. +44 1603 59 1796*

April 23, 2026

Researchers who classify text with LLMs often possess codebooks written for human annotators. We ask how these codebooks can serve as prompts. Using three codebooks from experimental economics (one on promise classification, two on strategic thinking), we vary the level of information in the prompt, along with surface variation such as formatting, framing, and wording, across two proprietary (GPT 5.4, GPT 5.4-nano) and two open-weight (Qwen 3.5 397B, Qwen 3.5 9B) models. Used as prompts, the codebooks reach 82–88% agreement with humans across the three tasks. On the recognition-heavy task (promises), model choice accounts for most of the variation in accuracy; on the learning-heavy tasks (strategic thinking), classification instructions carry comparable weight. We find that larger models make better use of additional information and are more robust to surface variation; while smaller models can be hurt by extra information and show some surface sensitivity without consistent direction. Codebook components and worked examples act as partial substitutes for one another, while model reasoning is neither a substitute for missing detail nor a complement once detail is provided. For researchers with an existing codebook, effort is better spent on the level of information in the prompt than on surface variations.

*Corresponding author.

1 Introduction

In text classification tasks, researchers typically need a codebook — either written from scratch to instruct human annotators, or adapted for a new setting from an existing one in the literature.¹ These codebooks define the categories, lay out the theoretical rationale, and work through annotated examples — all for human annotators. When researchers turn to large language models to perform the same task, a natural question arises: can the codebook prepared for human annotators be used directly as an LLM prompt? This matters for two audiences. The *practitioner* needs to know how much effort to invest: can the codebook be transcribed as-is, or must it be rewritten for the model? The *reviewer* needs to know what to pay attention to when assessing whether the classification is reliable: the formatting and framing of the prompt, or the level of informational detail with which the categories are defined?

The literature offers extensive guidance on how to prompt. Researchers are advised to structure their prompts with markdown headers and bullet lists (Han et al., 2025; Törnberg, 2024; Mishra et al., 2021b), to assign role personas (Dunivin, 2025; Carlson and Burbano, 2026; Kong et al., 2023), to provide background context and label descriptions (White et al., 2023; Peskine et al., 2023), to include worked examples (Brown et al., 2020), and to restructure the codebook for the model (Stuhler et al., 2025). Some of this advice draws on early prompting research on GPT-2 and GPT-3 (Reynolds and McDonell, 2021; Mishra et al., 2021a), where models were less capable and prompts were typically short task descriptions rather than detailed codebooks (Pecher et al., 2026). Following these recommendations requires substantial effort to transform the codebook into something “optimised” for the model, yet whether they still apply to today’s frontier models, or to substantially more detailed prompts, is unclear.

A separate concern is *prompt brittleness*. Apart from the informational content of a prompt, there is also the question of how that content is presented: choices about formatting (markdown vs. plain text, bullet lists vs. prose), framing (role persona, task preamble), and phrasing (specific word choices, sentence structure). We refer to these collectively as *surface variation*: changes that alter how the prompt reads without changing what it communicates about the classification task. Several studies have shown that LLM outputs can be sensitive to such variation, producing substantial changes in classification outcomes (Palmer et al., 2024; Sclar et al., 2024; Reiss, 2023). If models are sensitive to surface variations, then any particular prompt for-

¹For example, the promise classification codebook developed by Charness and Dufwenberg (2006) for a trust game was later adapted by Arad et al. (2024) for a public goods game.

mulation may not be robust, and iterating on phrasing and format may not lead to a reliably optimal prompting solution.

In this paper we take three classification codebooks from experimental economics, decompose each into its functional components — classification instructions, experiment context, theoretical background, and worked examples — and independently vary the level of informational detail in each alongside surface variations in a factorial design across two proprietary (GPT 5.4, GPT 5.4-nano) and two open-weight models (Qwen 3.5 397B, Qwen 3.5 9B).

Our central finding is that a codebook transcribed as the prompt, with minimal modification, reaches 82–88% agreement with humans across all four models tested. Accuracy² is driven by two components of the prompt: the level of informational detail in the classification instructions, and the worked examples. These two components partly substitute for one another. Both effects depend on the task and the model: on the learning-heavy tasks (strategic thinking), detailed instructions are essential, while on the recognition-heavy task (promises), model choice matters more than prompt content. Larger models (GPT 5.4 and Qwen 3.5 397B) benefit more from added detail, whereas additional detail can hurt smaller ones. Surface variations produce no significant effects for GPT 5.4 and broadly none for GPT 5.4-nano; the open-weight Qwen models are less robust to surface variation, but the direction of effects is inconsistent. Lastly, reasoning neither substitutes for missing detail nor adds value once detail is present.

The prompt brittleness reported in the literature appears to arise when either of two conditions holds: the prompt lacks sufficient classification detail, or the model is not large enough to be robust to surface variation. When either condition is relaxed — by providing a well-specified codebook or by using a model large enough to be robust to surface variation — brittleness largely disappears.

The practical recommendation is to provide a well-defined codebook with its worked examples and use a frontier model. The two bottlenecks are the level of informational detail provided to the model and the model’s capacity to process it effectively — not the way the information is packaged.

²Throughout the paper, *accuracy* (or *performance*) refers to the agreement rate between the model’s classification and the ground truth, defined as the label on which two independent human annotators agreed. See Section 4 for details.

2 Related Work

Large language models are increasingly used for text classification in the social sciences, with recent work confirming that LLMs can achieve inter-rater reliability comparable to trained coders across multiple disciplines (Gilardi et al., 2023; Ziems et al., 2024; Ornstein et al., 2025; Chae and Davidson, 2025). As adoption grows, so does the question of how best to instruct the model. Studies testing this question produce apparently conflicting recommendations: formatting matters substantially (Sclar et al., 2024; Han et al., 2025); formatting does not matter (Pangakis et al., 2023; Gilardi et al., 2023); detailed codebooks help (Lupo et al., 2024; Halterman and Keith, 2025; Atreja et al., 2025); detailed codebooks can hurt (Ruckdeschel, 2025, for small models); personas are recommended (Törnberg, 2024; Dunivin, 2025); personas are ineffective or harmful (Zheng et al., 2024; Kim et al., 2024; Hu et al., 2026; McLaren et al., 2026).

A useful organising principle comes from Pecher et al. (2026), who distinguish *underspecified* prompts — which provide little task description and no label constraints — from *well-specified* prompts that include explicit instructions.³ On underspecified prompts, model predictions are near-chance and any surface variation can shift outcomes; on well-specified prompts, predictions are confident and surface variations become noise. This distinction goes a long way toward reconciling the conflicting findings above: the studies that find large formatting, framing, and phrasing effects mostly test on underspecified prompts, small models, or both, while the studies that find minimal effects mostly use well-specified codebook-based prompts, larger models, or both. We do not claim this resolves the full literature — only that it is consistent with the results most directly comparable to ours.

The closest prior study in design is Reich et al. (2025), who cross eight binary prompt-strategy flags in a full factorial (864 combinations) and also find that classification instructions are the dominant component, while focusing on small open-weight models (Mistral 7B and Mistral NeMo 12B) and holding informational detail approximately constant within the classification instructions themselves.⁴

³Pecher et al. establish this distinction at the lower end of the specification range: their underspecified prompts are essentially raw completion cues of around five words (e.g., “{text}\nSentiment:”), while their well-specified prompts are brief task descriptions of around 25 words (e.g., “Determine the sentiment of the following text. Use one of these labels: negative; positive.”). Our design complements theirs by extending further into richer specification levels: our minimal prompt is already comparable to their well-specified condition, and our verbatim codebook prompts run to 1,000–1,500 words including theoretical frameworks, category definitions, and worked examples. Building on their finding that any task description beats no task description, we show that the further richness of a full researcher codebook continues to add value.

⁴A detailed comparison with further prior work appears in Table 24 (see Appendix S.12).

By crossing the level of informational detail with surface variation and model scale, we identify two interactions that prior work has not jointly tested: the level of informational detail determines whether surface variation matters at all, and model scale determines how much. This explains why prior studies, each testing one corner of this space, reached apparently conflicting conclusions.

3 Research Questions

In our setting, a codebook contains four functional components: classification instructions defining each category, experiment context describing the game mechanics, theory context laying out the formal theoretical framework, and worked examples. This structure is characteristic of experimental economics, where classifications are often grounded in a formal model. Codebooks in other domains may omit the theory layer, but classification instructions and examples are likely present in any well-specified codebook. How much of this detail a model actually needs depends on what it already knows from pre-training. Promise classification (P) is likely a *recognition-heavy* task: the concept is intuitive, and related notions of commitment and promise-keeping appear widely in natural language (Pan et al., 2023; Wang et al., 2024). Strategic thinking classification (L) is a *learning-heavy* task: level- k reasoning is a technical construct specific to behavioural game theory (Crawford et al., 2013), unlikely to be prevalent in pre-training corpora. We therefore ask:

RQ1. *Can existing codebooks serve as effective LLM prompts, and which of their components drive classification performance?*

Even if the codebook content is sufficient, the prompting literature makes strong recommendations on how to present it — formatting, persona assignment, reasoning — typically without controlling for the level of informational detail present in the prompt. We test each intervention at both high and low levels of informational detail. This allows us to ask whether interventions such as formatting or reasoning improve classification in their own right, or whether they only appear to help when the prompt lacks sufficient detail and the intervention partially fills that gap. This leads to our second question:

RQ2. *Do prompt engineering choices — formatting, framing, and reasoning — improve classification beyond what the codebook content provides?*

The answers to both questions may depend on the model. A large model may extract value from a verbose codebook that overwhelms a smaller one; a smaller model may

depend on prompt engineering choices that make no difference for a frontier model. We approximate model scale by total parameter count where it is disclosed; for the proprietary GPT models, where counts are undisclosed, we treat the ‘nano’ suffix as marking GPT 5.4-nano as smaller than GPT 5.4. We evaluate four models at two scale tiers within each family: GPT 5.4 and GPT 5.4-nano (proprietary), Qwen 3.5 397B and Qwen 3.5 9B (open-weight). This motivates our third question:

RQ3. *How does model scale affect classification performance and moderate the sensitivity to prompt content and design?*

4 Data

We use classification codebooks from three experimental economics studies spanning two substantive constructs. The *promise* task (P) classifies open-ended chat messages from a public goods game (Arad et al., 2024) as promise or empty talk; we use the 1,494 player-level utterances (drawn from 719 chat messages) on which both annotators agreed. The *level- k* tasks classify statements by strategic reasoning level: L_I classifies 493 messages from a voting game (Çelebi and Penczynski, 2023) into levels 0–3, and L_{II} classifies 851 messages from a set of coordination games (van Elten and Penczynski, 2020) into levels 0–5.

In all three tasks, two human annotators independently classified each item; ground truth is defined by the instances on which both annotators agreed, and an item is scored as correct if the model’s prediction matches this agreed label.⁵

5 Design

All classification tasks use a standardized prompt template (Figure 1), informed by existing prompting guidelines and empirical findings on effective prompt design.⁶ The template begins with a general task description and a role persona, followed by experiment context, theory context, and classification instructions drawn from the codebook. Worked examples from the codebook are included in the n -shot conditions. The experiments that follow test each dimension of the template and their interactions, with the

⁵For L_I and L_{II} , annotators were incentivised via a coordination game — paid based on agreement with each other — and informed of their disagreements between two rounds of classification. Houser and Xiao (2011) show that this incentive structure makes annotators more responsive to fine-grained changes in classification instructions. The P annotations were produced without such incentivisation.

⁶Each template component is motivated by specific findings in the prompting literature. See Appendix S.11 for the full rationale and citations.

```

# General Task
- Classify <X> in <E>

# Role Persona
- Act as a behavioural economist specialized in text
classification, concept <C> and game <G>

# Context

## Experiment
- <Game mechanics>
- <Experimental design / Decision process>
- ...

## Theory
- <Theoretical framework>
- ...

# Classification Task
- Classify <X> as <Y> given criteria: <Y1, Y2, ...>
- ...

# Classification Coding
- Code <X> as <Z>
- ...

# Examples
- <Example text> <classification>
- ...

# Constraints
- Provide your response using the JSON format specified below
- Do not provide explanations

# Output Format
{'classification': '<value>'}

```

Note: Theory subsection is included only for *L*-tasks. Examples are included only in *n*-shot conditions. All prompts use markdown formatting with itemized lists.

Figure 1: Prompt template.

exception of the output format sections (constraints, classification coding, and output format), which are held constant across all conditions.⁷

In total, across all experiments, approximately 703,000 classification requests were submitted to the API (703,253 calls; 702,809 successful; 0.06% error rate), covering 1,182 unique model–task–condition combinations across 4 models. Experiments were conducted between 24 March and 6 April 2026.

⁷Constraints and output format enforce structured JSON responses, which has been shown to improve classification accuracy (Tam et al., 2024) and lets the classification label be read directly from the response without any string parsing.

5.1 Codebook-to-Prompt Pipeline

We convert each human classification codebook into a set of LLM prompts through a four-step pipeline applied uniformly to all four tasks. All LLM-assisted steps use Claude Opus 4.6.

Step 1: Filter. The raw codebook is processed with a structured filtering prompt (see Appendix S.10.1) that removes content relevant only to human annotators in a laboratory setting: greetings, payment details, software instructions, data-entry guidance, meta-instructions, and deadline warnings. Classification-relevant content — experiment mechanics, category definitions, and worked examples — is retained verbatim. As a light reframing, second-person laboratory instructions (“Your task is to...”) are converted to third-person descriptions (“Each participant...”). No information is added or paraphrased beyond this reframing.

Step 2: Separate. The filtered codebook is manually divided into three named sections: *experiment context* (game mechanics, payoffs, communication protocol), *theory context* (the level- k framework, L-tasks only), and *classification instructions* (category definitions and assignment rules). Worked examples are extracted into a separate file for use in few-shot conditions. This separation allows each section to be independently varied in subsequent experiments.

Step 3: Compress. Each section is iteratively compressed to two intermediate levels (C1 at approximately 50–60% and C2 at approximately 25–30% of the verbatim word count) using a structured compression prompt (see Appendix S.10.2). The prompt is applied twice in sequence (verbatim $\rightarrow$ C1, C1 $\rightarrow$ C2) and instructs the model to only remove content — never to add, infer, or paraphrase — while preserving a task-specific list of key technical terms at every level. Table 2 shows that embedding cosine similarity to the verbatim text remains above 0.80 at all compression levels, and that all key terms are preserved across all tasks and all compression steps.

Step 4: Assemble. At runtime, prompt variants are assembled from their component sections. The assembler concatenates sections in a fixed order (framing header, experiment context, theory context, classification instructions, examples) and applies the requested content level and format treatment, producing a single system prompt. The text to classify is provided in the user prompt.⁸

⁸All calls use temperature 0, structured JSON output via task-specific schemas, and token-level log-probs. Full API configuration, retry logic, and provider details in Appendix S.8.

5.2 Models

We evaluate four models: two proprietary and two open-weight (Table 1). The GPT 5.4 pair (5.4 and 5.4-nano) represents a frontier closed-source family at two capability tiers; parameter counts are not disclosed. The Qwen 3.5 pair (397B and 9B) represents a state-of-the-art open-weight family at two known scales — 17B active parameters (MoE) and 9B (dense) — enabling a within-family scale comparison.

Model	Company	Total	Active	Architecture	Released	Provider
GPT 5.4	OpenAI	—	—	closed	Mar 2026	Azure OpenAI
GPT 5.4-nano	OpenAI	—	—	closed	Mar 2026	Azure OpenAI
Qwen 3.5 397B	Alibaba	397B	17B	MoE	Feb 2026	Together AI / OpenRouter
Qwen 3.5 9B	Alibaba	9B	9B	dense	Feb 2026	Together AI / OpenRouter

Note: Active params = parameters active per forward pass (equal to total for dense models). GPT parameter counts are not disclosed. GPT models support reasoning at multiple effort levels; Qwen models support a binary reasoning toggle.

Table 1: Models evaluated in this study.

GPT models are accessed through Azure OpenAI Service; Qwen models through Together AI and OpenRouter. Concurrent requests are distributed across multiple endpoints to maximise throughput.

5.3 Experiments

We conduct six experiments, each targeting a different dimension of prompt design. All experiments use the same items and models, enabling paired comparisons throughout. We represent each prompt configuration as a tuple (c, x, t, e) , where c is the classification instruction level, x is the experiment context level, t is the theory context level (L -tasks only), and e is the example condition. Subscripts denote detail: verbatim (V), compressed (C_1, C_2), or absent (N) for context and theory; verbatim (V) or simple (S) for classification instructions; zero-shot ($\emptyset$) or n -shot (n) for examples. Two anchoring configurations are used throughout:

- **Verbatim** ($c_V, x_V, t_V, \emptyset$): the full codebook content — classification instructions, context, and theory — transcribed verbatim, without examples.
- **Minimal** ($c_S, x_N, t_N, \emptyset$): one-line category descriptions only, with no context, theory, or examples.

5.3.1 Content Ablation

Each codebook is decomposed into three independently varied information sources: classification instructions (verbatim or simple), experiment context (V/C1/C2/N), and theory context (L -tasks only; V/C1/C2/N). Crossing these factors yields 8 variants per P -task (4×2 , no theory) and 32 per L -task ($4 \times 4 \times 2$).

Classification instructions are tested at two levels. *Verbatim* (V) transcribes the codebook’s category definitions in full — typically 6–15 bullet points per category covering inclusion criteria, edge cases, and boundary rules. *Simple* (S) reduces codebook’s category definitions to a single definitional sentence (e.g., “Classify the player’s level of strategic thinking as 0, 1, 2, or 3”), retaining only 3–11% of the verbatim word count.

Experiment context and theory context are each compressed to two intermediate levels using a structured compression prompt that enforces key-term preservation (see Appendix S.10.2). The prompt is applied iteratively: $V \rightarrow C1$ ($\approx 50\%$ of the original word count), then $C1 \rightarrow C2$ ($\approx 25\%$). At each step, the model is instructed to only remove content — never to add, infer, or paraphrase — while preserving a task-specific list of key technical terms. Table 2 confirms that embedding cosine similarity to the verbatim text remains above 0.80 at all compression levels and all key terms are retained. Full cosine similarities for all tasks are in Appendix S.1.3; the full factorial design is in Appendix S.6.1.

Source	Level	Cosine	Words	Key terms
Exp. context	V	1.000	293 (100%)	10
	C1	0.857	174 (59%)	10/10
	C2	0.831	97 (33%)	10/10
	N	—	0 (0%)	—
Theory	V	1.000	735 (100%)	9
	C1	0.901	374 (51%)	9/9
	C2	0.804	158 (21%)	9/9
	N	—	0 (0%)	—
Cls. instructions	V	1.000	—	—
	S	0.724	— (3%)	—

Note: Cosine similarity computed against the verbatim original using the `text-embedding-3-large` model of OpenAI. Key terms = task-critical tokens (game labels, payoff numbers, category names) enforced by the compression prompt. Word counts shown for L_1 ; other tasks in Appendix S.1.3.

Table 2: Semantic retention by prompt level (L_1).

5.3.2 Formatting and Framing

Format. Five visual treatments are applied to identical textual content: markdown with headers and bullet lists (baseline), titles with bullet lists, titles with prose paragraphs, flat bullet lists without headers, and plain unformatted text. Each format is tested at both verbatim and minimal content levels, yielding $5 \text{ formats} \times 3 \text{ tasks} \times 2 \text{ content levels} = 30$ conditions per model (see Appendix S.6.2 for details).

Framing. Two framing elements are tested: a general task description (GT) that names the experimental domain (e.g., “classify messages from a coordination game”) and a role persona (RP) that assigns a domain identity (e.g., “behavioral economist with expertise in level- k reasoning”). Three removal conditions (GT only, RP only, Neither) are compared against the full-framing baseline at both content levels, yielding $4 \times 3 \times 2 = 24$ conditions per model (see Appendix S.6.3 for details).

5.3.3 Examples

Each codebook’s worked examples are added to the prompt in an n -shot format. The number of examples is determined by what each codebook provides: 21 for P (10.5 per label), 13 for L_I (3.25 per level), and 5 for L_{II} (1.25 per level). Examples are tested at both content levels (minimal and verbatim), enabling a direct comparison of content versus examples as alternative sources of classification information, yielding $2 \text{ example conditions} \times 2 \text{ content levels} \times 3 \text{ tasks} = 12$ conditions per model.

5.3.4 Reasoning

Model reasoning is enabled at multiple effort levels. GPT models support five levels (off, low, medium, high, extra-high); Qwen models support a binary toggle (on/off). Reasoning is crossed with content level (minimal/verbatim) and example provision (0-shot/ n -shot), yielding $5 \times 2 \times 2 \times 3 = 60$ reasoning variants per GPT model and $2 \times 2 \times 2 \times 3 = 24$ per Qwen model (baselines shared with other experiments) (see Appendix S.6.4 for details).

5.4 Metrics

LLMs are not fully deterministic: even with the same prompt, the same input text, and temperature set to 0, repeated runs can produce different classifications. This arises from numerical non-associativity in floating-point operations during inference, which makes token selection sensitive to hardware-level variation when the gap between the top candidates is small (Yuan et al., 2025). In practice, studies running the same prompt

on the same items 30–50 times report that 8–12% of classifications change across runs, with more complex tasks showing even higher variability (Atil et al., 2024; Wang and Wang, 2025). A few-point accuracy gap between two prompts can therefore reflect run-to-run noise rather than a genuine treatment effect. We address this by using McNemar’s test throughout: each comparison pairs the *same items* under two conditions, and the test asks whether the items on which the two conditions disagree split asymmetrically, with one condition systematically more often correct than the other. This paired design controls for item difficulty and isolates the treatment effect from run-to-run variation. Significance is assessed at $p < 0.001$ throughout. Accuracy is computed per item as described in Section 4; all accuracy estimates are reported with 95% Wilson confidence intervals (Wilson, 1927). Additionally, instead of accuracy, Appendix S.13 reports tables and decompositions in terms of the support-weighted macro F1 score with paired bootstrap confidence intervals; the qualitative findings are unchanged, and the F1 replication is discussed there alongside the accuracy comparison.

6 Results

6.1 Content

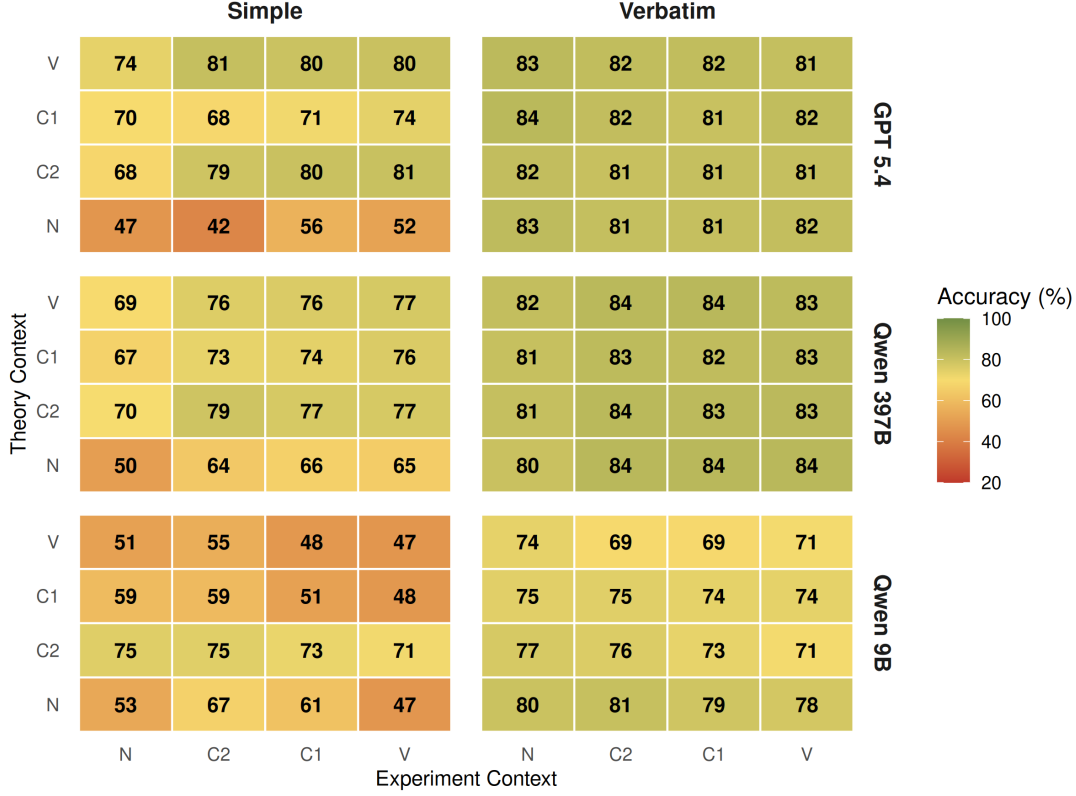
	P	Minimal		P	Verbatim	
		L_I	L_{II}		L_I	L_{II}
GPT 5.4	73.2 [70.9, 75.3]	47.4 [43.0, 51.8]	49.6 [46.2, 52.9]	74.2 [71.9, 76.3]	81.3 [77.7, 84.5]	81.0 [78.2, 83.5]
GPT 5.4-nano	60.9 [58.4, 63.4]	52.5 [48.1, 56.9]	53.1 [49.8, 56.5]	64.3 [61.8, 66.6]	70.2 [66.0, 74.1]	68.9 [65.7, 71.9]
Qwen 397B	70.5 [68.1, 72.7]	50.1 [45.7, 54.5]	43.6 [40.3, 46.9]	66.2 [63.8, 68.6]	83.0 [79.4, 86.0]	74.6 [71.6, 77.4]
Qwen 9B	64.1 [61.7, 66.5]	53.1 [48.7, 57.5]	38.8 [35.6, 42.1]	63.3 [60.8, 65.7]	70.8 [66.6, 74.6]	64.4 [61.1, 67.5]

Note: 95% Wilson CIs below each estimate. **Bold** = highest value per task within each content level; tied values (McNemar $p \geq 0.001$) share bold.

Table 3: Accuracy (%) at the minimal and verbatim prompts.

Table 3 reports accuracy under the minimal and the verbatim prompt. On P , the verbatim prompt yields no significant improvement over minimal: GPT 5.4 moves from 73.2% to 74.2%, and the other models show similarly small or even negative changes. Promise classification thus appears to be recognition-heavy: the model can often map

the label to the relevant concept without needing further information.⁹ The L -tasks, by contrast, are fundamentally learning-heavy. Under the minimal prompt, performance is poor but improves as the information needed to interpret the categories is provided.



Note: Accuracy (%) by experiment context (x-axis) and theory context (y-axis), separately under simple (left) and verbatim (right) classification instructions. Context levels: N = none, C2 $\approx$ 25%, C1 $\approx$ 50%, V = verbatim.

Figure 2: L_I information surface.

Figure 2 shows how theory and experiment context interact on L_I across different levels of classification detail (L_{II} surfaces in Appendix S.1.1). Under simple classification instructions in the left column, theory contributes substantially on its own, whereas experiment context alone either hurts performance or marginally benefits it. Once theory is present, however, experiment context becomes effective, indicating that the two components complement each other: theory helps define the categories, while experiment context helps map them onto the task. Together, they can partially substitute for missing classification instructions. By contrast, under verbatim classification instructions in the right column, the surface is largely flat for GPT 5.4 and Qwen 397B, with

⁹A variance decomposition (see Appendix S.1.2) confirms this: on P , model choice accounts for nearly all explained variance (98%), while on the L -tasks classification instructions dominate (58% on L_I).

only small positive differences across context conditions. This suggests that theory and experiment context matter primarily when classification instructions are under-specified.

Model scale further moderates this pattern. GPT 5.4 and Qwen 397B benefit from additional content when classification instructions are absent and still benefit marginally when classification instructions are present. Smaller models, by contrast, perform best when classification instructions are present but context is limited or absent, indicating that they are less able to make effective use of the full contextual information (per-model information surfaces in Appendix S.1.1).

Result 1. *Experiment and theory contexts partially complement each other and partially substitute for classification instructions. Larger models benefit from richer content, whereas smaller models perform better with less.*

6.2 Formatting and Framing

	Formatting				Framing			
	Minimal		Verbatim		Minimal		Verbatim	
	+	−	+	−	+	−	+	−
GPT 5.4	.17	.08	.00	.08	.33	.11	.33	.00
GPT 5.4-nano	.08	.00	.08	.00	.33	.22	.00	.00
Qwen 397B	.08	.08	.00	.00	.11	.44	.00	.00
Qwen 9B	.33	.17	.00	.00	.44	.00	.00	.11

Note: Proportion of all comparisons that are significantly positive (+) or negative (−) at $p < 0.001$. Formatting: 4 visual treatments vs. markdown baseline (12 per cell: 4 formats $\times$ 3 tasks). Framing: 3 conditions (both, GT only, RP only) vs. no-framing baseline (9 per cell: 3 conditions $\times$ 3 tasks). Mean effects and dispersion in Appendix S.2.

Table 4: Formatting and framing: proportion of significant effects.

Formatting. On verbatim prompts, switching from markdown to any of the four alternative formats produces almost no significant effects for any model (Table 4, left half). On minimal prompts, Qwen 9B shows scattered significance (.33 positive, .17 negative) but with no consistent direction; the other three models remain largely unaffected. *Framing.* On verbatim prompts, framing effects are again negligible (Table 4, right half). On minimal prompts, the pattern is model-dependent: Qwen 9B shows .44 significant positive effects (driven by domain cues in the general task description that are absent from the sparse prompt), while Qwen 397B shows .44 significant negative.

For GPT 5.4, the few significant framing effects are driven entirely by the general task description (GT) rather than the role persona.¹⁰ See Appendix S.2 for full per-model effect sizes, dispersion, and per-condition significance tables.

Result 2. *Neither formatting nor framing reliably affects classification accuracy. On verbatim prompts, effects are negligible. On minimal prompts, significance increases but the direction is inconsistent across models and tasks.*

6.3 Examples

	P	L_I	L_{II}
Δ_{Ex}	+16.6*	+37.4*	+20.4*
$\Delta_{\text{Ex}} - \Delta_{\text{Content}}$	+15.6*	+3.4	-10.9*
$\Delta_{\text{Ex} \text{Content}}$	+14.0*	+2.6	+0.7
$\Delta_{\text{Content} \text{Ex}}$	-1.6	-0.8	+11.6*

Note: * = McNemar $p < 0.001$. All values in pp. Δ_{Ex} : gain from adding examples to the minimal prompt. $\Delta_{\text{Ex}|\text{Content}}$: gain from adding examples to the verbatim prompt. $\Delta_{\text{Ex}} - \Delta_{\text{Content}}$: minimal+examples minus verbatim+zero-shot (positive = examples alone outperform content alone). $\Delta_{\text{Content}|\text{Ex}}$: verbatim+examples minus minimal+examples (positive = content adds value on top of examples).

Table 5: GPT 5.4: effect of examples.

Table 5 decomposes the contributions of content and examples¹¹ for GPT 5.4. Examples alone produce large accuracy gains on every task ($\Delta_{\text{Ex}} = +16.6$ pp on P , +37.4 pp on L_I , +20.4 pp on L_{II}), comparable to or larger than the gain from adding verbatim content alone. Content and examples convey overlapping information about what the categories mean, making them partial substitutes. Once verbatim content is already in the prompt, the value of additional examples scales with the number of examples per label the codebook supplies¹²: $\Delta_{\text{Ex}|\text{Content}}$ is +14.0 pp on P , +2.6 pp on L_I , and a

¹⁰On verbatim prompts, GPT 5.4 shows three significant positive effects, all GT-driven: GT only on P (+2.9 pp), GT only on L_{II} (+3.6 pp), and Both on P (+2.8 pp). The GT sentence names the domain (“public goods game”, “coordination game”), providing a keyword anchor even when the full codebook is present. Role persona alone never reaches significance on any model on verbatim prompts. See Appendix S.2.1 for the full per-model significance counts broken down by framing condition (GT only, RP only, both, neither).

¹¹We also varied the in-prompt format of the examples (markdown headers, XML tags, plain delimiters, arrow notation, multi-turn conversation); design and full results in Appendix S.3.4.

¹² P provides 21 worked examples (10.5 per label), L_I 13 (3.25 per level), and L_{II} 5 (1.25 per level); see Appendix S.3.1. A task-intrinsic effect — P being more pattern-recognition and L more theory-

non-significant +0.7 pp on L_{II} . The same monotonic pattern appears under weighted F1 and across all four models.¹³

Result 3. *Examples alone produce large accuracy gains across all tasks. Content and examples are partial substitutes. The accuracy gain from examples given content scales with the example count.*

Table 6 represents what a researcher obtains by faithfully transcribing the codebook with its worked examples (with the minor LLM-assisted filtering described in Section 5.1). On P and L_I , the open-weight Qwen 397B matches the proprietary GPT 5.4 (McNemar $p \geq 0.001$), and even Qwen 9B reaches 83% on L_I . The tasks separate clearly by difficulty: all models achieve 72–88% on P and 78–84% on L_I , but L_{II} drops to 68–82%, with the gap between frontier and small models widening from ≈ 2 pp on L_I to ≈ 13 pp on L_{II} .

	P	L_I	L_{II}
GPT 5.4	88.2 [86.4, 89.7]	84.0 [80.5, 86.9]	81.7 [78.9, 84.1]
GPT 5.4-nano	72.0 [69.6, 74.2]	78.3 [74.4, 81.7]	70.7 [67.6, 73.7]
Qwen 397B	86.7 [84.9, 88.4]	83.6 [80.0, 86.6]	77.8 [74.9, 80.5]
Qwen 9B	78.5 [76.4, 80.5]	82.8 [79.2, 85.8]	68.2 [64.9, 71.2]

Note: Verbatim content + n -shot examples, no reasoning. 95% Wilson CIs below each estimate.

Bold = highest value per task; tied values (McNemar $p \geq 0.001$) share bold.

Table 6: Accuracy (%) with the full codebook: verbatim content and worked examples.

6.4 Reasoning

Can reasoning compensate for missing codebook content — that is, can a model that “thinks harder” overcome an underspecified prompt? Table 7 decomposes reasoning and content effects in the same way Table 5 decomposed examples and content.

laden — may also contribute to this monotonic decline; our design does not let us isolate it from the example-count effect.

¹³Weighted-F1 replication in Appendix S.13; per-model decomposition in Appendix S.3.2. L_I comparison ($\Delta_{\text{Content alone}} \rightarrow \Delta_{\text{Content|Ex}}$): Qwen 9B +17.7 $\rightarrow$ +17.3 pp (essentially additive), GPT 5.4-nano +17.7 $\rightarrow$ +5.7 pp (partial substitution), Qwen 397B +32.9 $\rightarrow$ +1.4 pp (full substitution), GPT 5.4 +34.0 $\rightarrow$ -0.8 pp (full substitution). Substitution thus tracks scale rather than open/closed status: the 397B open model behaves like the proprietary frontier, while the 9B open model is the outlier. Qwen 9B itself reverts to substitution on L_{II} once the full codebook saturates (+25.6 $\rightarrow$ -1.2 pp).

The answer is no. On L -tasks, content alone clearly outperforms reasoning alone ($\Delta_{\text{Rea}} - \Delta_{\text{Content}}$: -45.4 pp on L_I , -11.6 pp on L_{II}). On L_I , reasoning on the minimal prompt actually *hurts*: $\Delta_{\text{Rea}} = -11.5$ pp, meaning that the model’s attempt to reason without adequate classification instructions produces worse results than no reasoning at all. The conditional effects are asymmetric: content remains highly valuable when reasoning is active ($\Delta_{\text{Content}|\text{Rea}}$: $+46.7$ pp on L_I , $+11.6$ pp on L_{II}), but reasoning adds nothing when content is already present ($\Delta_{\text{Rea}|\text{Content}}$: $+1.2$ pp n.s. on L_I , exactly 0.0 pp on L_{II}). The same pattern holds across all four models (see Appendix S.4.3): $\Delta_{\text{Rea}|\text{Content}}$ is near-zero on L -tasks for GPT 5.4, GPT 5.4-nano, and Qwen 397B, with only Qwen 9B showing a significant effect on L_I ($+10.5$ pp). On P , smaller models show significant reasoning-given-content gains (GPT 5.4-nano: $+6.5$ pp, Qwen 397B: $+8.4$ pp, Qwen 9B: $+9.4$ pp), suggesting that reasoning can partially compensate for weaker pre-training representations of familiar concepts, but not for missing domain-specific theory.

	P	L_I	L_{II}
Δ_{Rea}	$+9.5^*$	-11.5^*	$+19.7^*$
$\Delta_{\text{Rea}} - \Delta_{\text{Content}}$	$+8.5^*$	-45.4^*	-11.6^*
$\Delta_{\text{Content} \text{Rea}}$	-9.7^*	$+46.7^*$	$+11.6^*$
$\Delta_{\text{Rea} \text{Content}}$	-1.2	$+1.2$	$+0.0$

Note: * = McNemar $p < 0.001$. All values in pp, 0-shot only, no examples. For GPT models we test low, medium, high, and extra-high reasoning effort; we report high to match Qwen’s reasoning token output (≈ 750 tokens). No incremental step from low to extra-high produces a significant improvement (see Appendix S.4.1). Δ_{Rea} : minimal+reasoning minus minimal 0-shot. $\Delta_{\text{Rea}} - \Delta_{\text{Content}}$: minimal+reasoning minus verbatim 0-shot (negative = content alone outperforms reasoning alone). $\Delta_{\text{Content}|\text{Rea}}$: verbatim+reasoning minus minimal+reasoning. $\Delta_{\text{Rea}|\text{Content}}$: verbatim+reasoning minus verbatim 0-shot. Remaining models in Appendix S.4.3.

Table 7: GPT 5.4: content vs. reasoning substitutability.

With reasoning enabled on verbatim prompts at both 0-shot and n -shot (per-model effects in Appendix S.4.2), GPT 5.4 shows no significant effect on any task at any condition. Smaller models benefit on P at both shot conditions, but on L -tasks their gains appear only at 0-shot and disappear once examples are present: examples render the reasoning gains redundant.¹⁴

¹⁴Adding examples costs $\approx 0.3\text{--}0.4 \times$ the prompt (input-rate tokens), comparable to low reasoning, yet produces accuracy gains an order of magnitude larger on L -tasks. See Appendix S.9 for the full cost comparison.

Result 4. *Reasoning cannot substitute for codebook content and does not improve frontier-model classification. On well-specified prompts, smaller models gain from reasoning only on the P-task; their L-task gains at zero-shot disappear once examples are present.*

7 Discussion

7.1 Summary of Findings

Faithful transcription of a codebook, including category definitions, context, theory, and the worked examples it supplies, produces 82–88% agreement with the human-agreed ground truth across our three tasks. What drives this accuracy is the classification-relevant information in the prompt: detailed classification instructions are the dominant component, while experiment and theory contexts partially complement each other and together can partially substitute for the classification instructions. Examples on their own also produce large gains. The information they carry partially overlaps with that of the codebook content, making the two partial substitutes: once a verbatim codebook is in place, the value of additional examples scales with how many examples per label the codebook supplies.

Surface variations do not reliably improve on the transcribed codebook. Formatting, framing, phrasing, and reasoning produce negligible effects when the codebook is faithfully transcribed; reasoning in particular cannot compensate for missing content and adds little once content is present.

How much of this robustness holds depends on model scale. Larger models tolerate more surface variation and extract more value from detailed content; smaller models are more sensitive, but the direction of their sensitivity is inconsistent across tasks and treatments, so prompt engineering does not become a more reliable lever for them either.

7.2 Beyond the Codebook

A researcher adapting a human codebook for an LLM might feel that the original is too sparse in places and want to elaborate, adding the kind of explanation, rationale, or worked-out criteria that a careful human reader would also benefit from. We systematically expanded the codebook in this way¹⁵ and saw no significant change for

¹⁵We tested three perturbations of the prompt text itself: (i) verbose elaboration (+50–275% words of relevant detail) plus a matched noise control of irrelevant content at the same length; (ii) round-trip back-translation (English → Turkish/German → English); (iii) the raw unprocessed codebook

GPT 5.4; injecting irrelevant content of matched length had no effect either, suggesting that frontier models filter out what is not useful. Smaller models respond to both treatments at 0-shot, but the effects largely vanish once examples are present. Similarly, paraphrasing produces effects below 1 pp for all models at 0-shot; at n -shot, smaller models show occasional sensitivity, again without consistent direction. And the raw unprocessed codebook, including payment instructions and software notes intended for human annotators, performs comparably to the cleaned version for GPT 5.4.

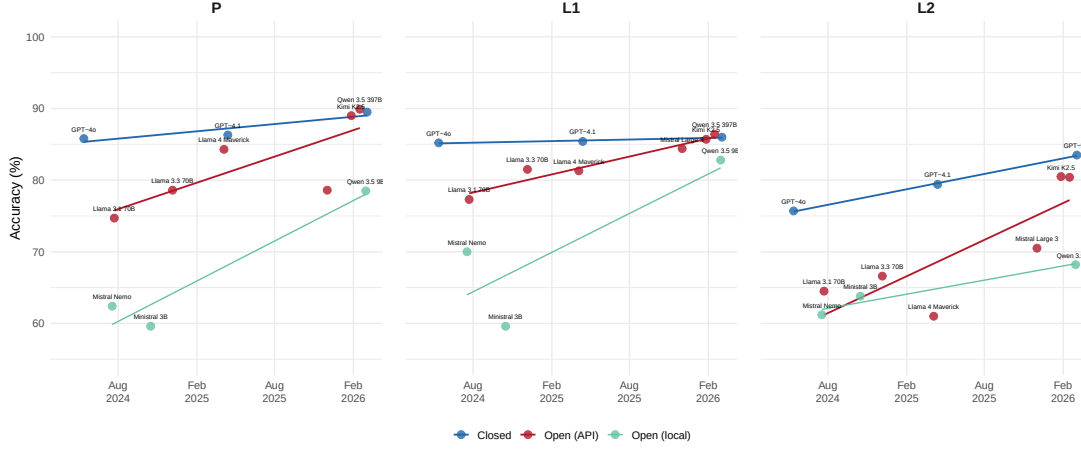
As Sections 1 and 2 discuss, the apparently conflicting reports of prompt brittleness in the literature largely correspond to underspecified prompts or weaker models. Our results extend the point beyond formatting: verbose elaboration, irrelevant noise, paraphrasing, and uncleaned procedural content all fail to move the needle once the codebook is detailed and the model is capable.

A parallel observation comes from compound AI systems: Zhang et al. (2026) find that prompt optimisation is already a coin flip on tasks where the model’s default behaviour is well-aligned, and argue that instruction tuning and reinforcement learning from human feedback increasingly compress diverse input phrasings into consistent outputs, shrinking the headroom that surface-level optimisation can exploit. We see the same shrinking headroom in the classification setting: much of the prompt-engineering advice in the literature was written for earlier, less capable models, and the domain in which it still moves the needle has been narrowing as models improve.

This robustness reflects the quality of our codebooks: experimental-economics codebooks define categories through formal models, give inclusion and exclusion criteria, and provide worked examples grounded in the same theory. Codebooks elsewhere in the social sciences are often sparser, with many labels, short definitions, overlapping categories, and no formal theoretical grounding. In those settings the codebook may genuinely be underspecified, and elaboration or restructuring may be warranted. Our content-ablation results (Section 6.1) make the asymmetry concrete: removing detail from an already-detailed codebook drops accuracy sharply on theory-dependent tasks, so the model is not indifferent to content — only to surplus content. For researchers in other domains, the practical question is therefore not whether to invest time in prompt engineering, but whether to invest it in enriching the codebook itself — adding the category definitions, boundary cases, and worked examples that our codebooks already provide.

This raises a diagnostic question: how can a researcher tell whether their prompt

used directly as the prompt. Designs in Appendix S.6.5; significance tables in Appendix S.5 (verbose/noise) and Appendix S.5.2 (paraphrase).



Note: Accuracy on verbatim n -shot prompts (with reasoning where available) across model generations. Blue = proprietary GPT family; red = frontier open-weight models; green = small open-weight models ($\leq 10B$) that can run on consumer hardware.

Figure 3: Model performance over time by task.

is already well-specified or still underspecified? Our results suggest that robustness to surface variation is itself the signal: a well-specified prompt produces consistent classifications regardless of how it is formatted, framed, or paraphrased, while an underspecified prompt does not. Paraphrasing is one such surface variation, and it is exactly how Barrie et al. (2025) measure prompt stability. What their framework does not address is why one prompt is stable and another is not. Our results point to a candidate explanation: stability emerges once the prompt saturates the information the model can use, after which further content or rewording yields only marginal gains. Practical tests for when a given prompt and model have reached this saturation point are a natural direction for future work.

7.3 Open or Closed Models

Figure 3 plots accuracy over time for models featured in the prompting literature we cite, alongside additional models we evaluated to broaden the comparison.¹⁶ Three trends are visible. First, frontier open-weight models (red) have converged toward proprietary performance (blue) over two years, with the steepest gains on L -tasks: Qwen 3.5 397B now matches GPT 5.4 on P and L_1 . Second, small open-weight models that can run on consumer hardware (green, $\leq 10B$ parameters) are also improving

¹⁶Beyond the four models analysed in this paper, we ran the same verbatim n -shot classification on GPT 4o, GPT 4.1, Llama 3.1 70B, Llama 3.3 70B, Llama 4 Maverick, Kimi K2.5, Mistral Large 3, Mistral NeMo (12B), and Minstral 3B. These additional runs are used here for the timeline comparison only; the factorial experiments reported in the results are limited to the four featured models.

rapidly — Qwen 3.5 9B reaches 82–84% on L_I , a level that required frontier models a generation earlier. Third, GPT 5.4 retains a clear lead on the hardest task (L_{II}), where the gap between proprietary and open models is still substantial.

A closing accuracy gap does not mean open models are equally dependable in practice. Smaller open-weight models remain more sensitive to surface variations: formatting, framing, and wording changes that have no effect on GPT 5.4 can produce significant shifts for Qwen 9B (Section 6.2). This matters for the distinction between *reproducibility* (repeating the same analysis with the same prompt and model) and *replicability* (arriving at the same conclusions with reasonable variations in prompt design). A surface-sensitive open model may reproduce well¹⁷ — the same prompt and weights yield the same outputs — yet fail to generalise when another researcher adapts the prompt to a new setting and minor formatting, framing, or phrasing changes shift the results. GPT 5.4’s insensitivity to these choices makes it more replicable, not just more accurate.

The main argument against proprietary models is reproducibility: if the provider deprecates the model, the exact same classification can no longer be reproduced. But how bad is this problem in practice? Inter-model agreement across three consecutive GPT generations (4o, 4.1, 5.4) on identical items is high.¹⁸ A researcher who classified data with GPT 4o and later re-ran the analysis with GPT 5.4 would obtain higher accuracy, potentially greater robustness to prompt variation, and classification agreement comparable to substituting one trained human annotator for another.¹⁹ This is not perfect reproducibility, but it is a predictable and improving trajectory — and no worse than the inter-coder reliability one would accept when replacing one human annotator with another.

Even our largest open-weight model, Qwen 3.5 397B, remains sensitive to surface-level prompt variation in a way that GPT 5.4 is not (Section 6.2). Reaching frontier-level robustness therefore appears to require a further scale jump for open models, plausibly to the trillion-parameter range that closed frontier models such as GPT 5.4

¹⁷No LLM classification is perfectly reproducible: even with identical prompts, temperature 0, and the same model, run-to-run variation produces flip rates of 2–12% depending on the model and task (Atil et al., 2024; Wang and Wang, 2025). We use “reproduce” here in the weaker sense of obtaining substantively similar results.

¹⁸We measure inter-model agreement using Krippendorff’s α , a reliability coefficient that accounts for chance agreement; $\alpha = 1$ indicates perfect agreement, and values above $\alpha \approx 0.7$ are commonly read as substantial agreement (see e.g. Barrie et al., 2025). On our data, adjacent GPT generations reach $\alpha = 0.89$ – 0.91 on P and 0.84 – 0.88 on L_I ; the full table is in Appendix S.7.

¹⁹For reference, on the P task the two human annotators in our source data (Arad et al., 2024) reach Krippendorff’s $\alpha = 0.48$ before disagreements were resolved — considerably below the cross-generation GPT agreement of 0.89 – 0.91 reported above.

are believed to inhabit. On the open side, Moonshot’s Kimi K2.5 currently exemplifies that range.²⁰

7.4 Limitations

Experimental-economics text is a best-case setting for codebook-based classification, and our results should be read with this in mind. Three features of our setting are unlikely to hold jointly in other social-science domains. Beyond the codebook quality discussed in Section 7.2, the text we classify is tightly constrained: subjects communicate within fixed experimental rules with known payoffs and structured protocols, producing a narrow vocabulary and a clear mapping between language and the concepts being classified. Tweets, parliamentary speeches, Reddit threads, or interview transcripts are far more variable in style, length, and ambiguity, and a model that ignores surface variations in our setting may not do so once the text itself becomes more variable. We are also able to provide the full experimental and theoretical context in the prompt because we designed the experiments; most applied settings lack this, leaving the researcher with a codebook and the text but no complete description of the data-generating process.

These conditions make our null finding on surface features a strong result within this setting; whether the same holds in domains where any of these conditions is relaxed remains an open question. The content-ablation results show what happens when prompt detail drops: accuracy on L -tasks falls from above 80% to below 50% when classification instructions are reduced to one line. Researchers working with codebooks closer to our minimal condition than our verbatim condition, as many social-science codebooks are, may find that the prompt design choices we show to be irrelevant here do matter for them. This can stem from two sources: their codebook may not yet give the model enough information to ignore how it is presented, or the concepts being classified may be intrinsically more ambiguous, limiting what any codebook can fully specify.

8 Conclusion

For frontier models on codebook-based classification tasks, the practical recommendation is clear: transcribe the codebook faithfully, include the worked examples it pro-

²⁰Kimi K2.5 was the only other state-of-the-art open-weight model released as recently as the Qwen 3.5 series at the time of writing. We did not include it in our factorial experiments because its non-reasoning mode is forced to temperature 0.6, and we did not want a model whose outputs come from a temperature level other than 0 to be mixed with the rest of our temperature-0 runs.

vides, and use a capable model. This yields 82–88% agreement with humans across the four models we test. Detailed classification instructions and worked examples are the two drivers of accuracy: partial substitutes that convey overlapping information about the categories, with classification instructions alone accounting for 58% of explained variance on level- k tasks. Surface variations (formatting, framing, wording, verbosity) are noise when the prompt is well-specified, and reasoning neither substitutes for missing detail nor adds value once detail is present. For smaller open-weight models, the same content-first principle applies but with care: verbose theory context that is harmless for frontier models can hurt smaller ones.

These findings suggest a shift in attention from surface prompt engineering toward two factors that actually matter: the level of informational detail of the classification information provided to the model, and the choice of a model capable of processing that information reliably. The effort conventionally allocated to formatting, persona assignment, and iterative wording refinement is better invested in ensuring that the classification instructions are clear and complete.

References

- Arad, Ayala, David Hugh-Jones, and Stefan P. Penczynski**, “Communication is informative about cooperation,” Technical Report, University of East Anglia 2024. Working paper.
- Atil, Berk, Aleyna Aykent, Breck Chittams, Jesse Baldwin, and Aylin Caliskan**, “Non-Determinism of “Deterministic” LLM Settings,” *arXiv preprint arXiv:2408.04667*, 2024.
- Atreja, Shubham, Joshua Ashkinaze, Lingyao Li, Julia Mendelsohn, and Libby Hemphill**, “Prompt Design Matters for Computational Social Science Tasks but in Unpredictable Ways,” in “Proceedings of the International AAAI Conference on Web and Social Media,” Vol. 19 2025, pp. 122–145.
- Barrie, Christopher, Evangelia Palaiologou, and Petter Törnberg**, “Prompt Stability Scoring for Text Annotation with Large Language Models,” *arXiv preprint arXiv:2407.02039*, 2025.
- Brown, Tom, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell et al.**, “Language models are few-shot learners,” *Advances in neural information processing systems*, 2020, 33, 1877–1901.
- Carlson, Kevin D and Vanessa C Burbano**, “The Use of LLMs to Annotate Data in Management Research,” *Strategic Management Journal*, 2026, 47 (3), 699–725.
- Çelebi, Can and Stefan P. Penczynski**, “Strategic Thinking in Jury Decisions: An Experimental Study,” Technical Report, School of Economics, University of East Anglia, Norwich, UK. 2023.
- Chae, Youngjin and Thomas Davidson**, “Large language models for text classification: From zero-shot learning to fine-tuning,” *Sociological Methods & Research*, 2025.
- Charness, Gary and Martin Dufwenberg**, “Promises and partnership,” *Econometrica*, 2006, 74 (6), 1579–1601.
- Crawford, Vincent P., Miguel A. Costa-Gomes, and Nagore Iriberri**, “Structural Models of Nonequilibrium Strategic Thinking: Theory, Evidence, and Applications,” *Journal of Economic Literature*, September 2013, 51 (1), 5–62.
- Dunivin, Zackary**, “Scaling Hermeneutics: Qualitative Coding with LLMs,” *EPJ Data Science*, 2025.
- Gilardi, Fabrizio, Meysam Alizadeh, and Maël Kubli**, “ChatGPT outperforms crowd workers for text-annotation tasks,” *Proceedings of the National Academy of*

Sciences, 2023, 120 (30), e2305016120.

Halterman, Andrew and Katherine A Keith, “Codebook LLMs: Evaluating LLMs as Measurement Tools for Political Science Concepts,” *Political Analysis*, 2025.

Han, Yuchen, Yucheng Wu, and Jeffrey Willard, “Effect of Selection Format on LLM Performance,” *arXiv preprint arXiv:2503.06926*, 2025.

Houser, Daniel and Erte Xiao, “Classification of natural language messages using a coordination game,” *Experimental Economics*, 2011, 14, 1–14.

Hu, Yifan, Mohammad Rostami, and Jesse Thomason, “Expert Personas Improve LLM Alignment but Damage Accuracy,” *arXiv preprint arXiv:2603.18507*, 2026.

Kim, Do Yeon, Woo-Jin Lee, Youngho Kim, and Joonsuk Kim, “Persona is a Double-edged Sword,” *arXiv preprint arXiv:2408.08631*, 2024.

Kong, Aobo, Shiwan Zhao, Hao Chen, Qicheng Li, Yong Qin, Ruiqi Sun, and Xin Zhou, “Better zero-shot reasoning with role-play prompting,” *arXiv preprint arXiv:2308.07702*, 2023.

Lupo, Luca, Måns Magnusson, Dirk Hovy, Elin Naurin, and Lena Wangnerud, “Towards Human-Level Text Coding with LLMs: The Case of Fatherhood Roles in Public Policy Documents,” *arXiv preprint arXiv:2311.11844*, 2024.

McLaren, Lorcan, James P. Cross, Zuzanna Krakowska, Robin Rauner, and Martijn Schoonvelde, “Magic Words or Methodical Work? Challenging Conventional Wisdom in LLM-Based Political Text Annotation,” *arXiv preprint arXiv:2603.26898*, 2026.

Mishra, Swaroop, Daniel Khashabi, Chitta Baral, and Hannaneh Hajishirzi, “Cross-task generalization via natural language crowdsourcing instructions,” *arXiv preprint arXiv:2104.08773*, 2021.

___, ___, ___, **Yejin Choi, and Hannaneh Hajishirzi**, “Reframing Instructional Prompts to GPTk’s Language,” *arXiv preprint arXiv:2109.07830*, 2021.

Ornstein, Joseph T, Eliot N Blasingame, and Jake S Truscott, “How to Train Your Stochastic Parrot: Large Language Models for Political Texts,” *Political Science Research and Methods*, 2025.

Palmer, Alexis, Noah A Smith, and Arthur Spirling, “Using proprietary language models in academic research requires explicit justification,” *Nature Computational Science*, 2024, 4 (1), 2–3.

Pan, Jane, Tianyu Gao, Howard Chen, and Danqi Chen, “What In-Context

Learning” Learns” In-Context: Disentangling Task Recognition and Task Learning,” *arXiv preprint arXiv:2305.09731*, 2023.

Pangakis, Nicholas, Samuel Wolken, and Neil Fasching, “Automated annotation with generative ai requires validation,” *arXiv preprint arXiv:2306.00176*, 2023.

Pecher, Branislav, Marek Spiegel, Jakub Belanec, and Jan Cegin, “Revisiting Prompt Sensitivity in Large Language Models for Text Classification: The Role of Prompt Underspecification,” *arXiv preprint arXiv:2602.04297*, 2026.

Peskine, Youri, Damir Korenčić, Ivan Grubisic, Paolo Papotti, Raphael Troncy, and Paolo Rosso, “Definitions Matter: Guiding GPT for Multi-label Classification,” in “Findings of the Association for Computational Linguistics: EMNLP 2023” 2023, pp. 4054–4063.

Reich, Andreas, Claudia Thoms, and Tobias Schrimpf, “Introducing HALC: A General Pipeline for Finding Optimal Prompting Strategies for Automated Coding with LLMs in the Computational Social Sciences,” *arXiv preprint arXiv:2507.21831*, 2025.

Reiss, Michael V, “Testing the reliability of chatgpt for text annotation and classification: A cautionary remark,” *arXiv preprint arXiv:2304.11085*, 2023.

Reynolds, Laria and Kyle McDonell, “Prompt programming for large language models: Beyond the few-shot paradigm,” in “Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems” 2021, pp. 1–7.

Ruckdeschel, Mattes, “Just Read the Codebook! Make Use of Quality Codebooks in Zero-Shot Classification of Multilabel Frame Datasets,” in “Proceedings of the 31st International Conference on Computational Linguistics (COLING)” 2025, pp. 6317–6337.

Sclar, Melanie, Yejin Choi, Yulia Tsvetkov, and Alane Suhr, “Quantifying Language Models’ Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting,” in “International Conference on Learning Representations (ICLR)” 2024.

Stuhler, Oscar, Adrien Ton, and Étienne Ollion, “From Codebooks to Promptbooks,” *Sociological Methods & Research*, 2025, 54, 794–848.

Tam, Zhi Rui, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung yi Lee, and Yun-Nung Chen, “Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models,” in “Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track” 2024.

Törnberg, Petter, “Best Practices for Text Annotation with Large Language

Models,” *Sociologica*, 2024, 18 (2), 67–85.

van Elten, Jonas and Stefan P Penczynski, “Coordination games with asymmetric payoffs: An experimental study with intra-group communication,” *Journal of Economic Behavior & Organization*, 2020, 169, 158–188.

Wang, Shucheng and Heather Wang, “Assessing Consistency and Reproducibility in the Outputs of Large Language Models,” *arXiv preprint arXiv:2503.16974*, 2025.

Wang, Xiaolei, Xinyu Tang, Wayne Xin Zhao, and Ji-Rong Wen, “Investigating the Pre-Training Dynamics of In-Context Learning: Task Recognition vs. Task Learning,” *arXiv preprint arXiv:2406.14022*, 2024.

White, Jules, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt, “A prompt pattern catalog to enhance prompt engineering with chatgpt,” *arXiv preprint arXiv:2302.11382*, 2023.

Wilson, Edwin B., “Probable Inference, the Law of Succession, and Statistical Inference,” *Journal of the American Statistical Association*, 1927, 22 (158), 209–212.

Yuan, Zhenyuan, Jiawei Jia, Fei Wang, Jiaxin Chen, Wotao Yin, and Tianyi Cai, “Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference,” *arXiv preprint arXiv:2506.09501*, 2025. NeurIPS 2025 Oral.

Zhang, Xing, Guanghui Wang, Yanwei Cui, Wei Qiu, Ziyuan Li, Bing Zhu, and Peiyang He, “Prompt Optimization Is a Coin Flip,” *arXiv preprint arXiv:2604.14585*, 2026.

Zheng, Mingqian, Jiaxin Pei, and David Jurgens, “When “A Helpful Assistant” Is Not Really Helpful: Personas in System Prompts Do Not Improve Performances of Large Language Models,” in “Findings of the Association for Computational Linguistics: EMNLP 2024” 2024.

Ziems, Caleb, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang, “Can large language models transform computational social science?,” *Computational Linguistics*, 2024, pp. 1–55.